

TI Summary:

Alphabete, Wörter, Sprachen

- ☐ Alphabet: Eine endliche, nichtleere Menge von Symbolen.
 - ☐ Wort: Eine endliche Folge von Symbolen aus einem Alphabet Σ .
- ☐ leeres Wort: λ
 - ☐ Die Länge eines Wortes x ist $|x|$
 - ☐ Σ^* bezeichnet die Menge aller Wörter über Σ
 - ☐ Konkatenation: xy
 - ☐ Umkehrung: x^R
 - ☐ Iteration: $x^0 = \lambda$, $x^i = x \cdot x^{i-1}$
 - ☐ Zählfunktionen: $|x|_a$ gibt die Anzahl des Symbols a in x an.
 - ☐ Vereinigung ($L_1 \cup L_2$)
 - ☐ Schnitt ($L_1 \cap L_2$)
 - ☐ Konkatenation ($L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$)
 - ☐ $L^* = \cup_n L^n = \{\lambda, \text{Wörter durch Wiederholung von } L\}$

Kolmogorov Komplexität

- ☐ Die Kolmogorov-Komplexität $K(x)$ eines Wortes $x \in \{0,1\}^*$ ist die minimale Länge (in Bit) eines Programms (in Pascal), das x generiert.
 - ☐ $K(x) \leq \log_2(|x|) + d$
 - ☐ $K(x) \geq |x| \Rightarrow$ Zufälliges Wort
- ☐ Maximale Anzahl Programme der Länge kleiner **gleich** x :

$$\sum_{k=1}^x 2^k = 2^{x+1} - 2$$

- ☐ Maximale Anzahl Programme der Länge kleiner x :

$$\sum_{k=1}^{x-1} 2^k = 2^x - 2$$

nach Pigeonhole Principe kann man dann zeigen, dass es mindestens eine Zahl geben muss die mindestens eine gewisse Kolmogorov-Komplexität hat

Im Intervall $[2^n, 2^{(n+1)}]$ hat es $2^n + 1$ Zahlen

Endliche Automaten (EA)

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : endliche Zustandsmenge

Σ : Eingabealphabet

δ : Übergangsfunktion $\delta : Q \times \Sigma \rightarrow Q$

q_0 : Startzustand

F : Menge der akzeptierenden Zustände

Schritt:

Ein **Schritt** von M ist eine Relation (auf Konfigurationen) $\xrightarrow{M} \subseteq (Q \times \Sigma^*) \times (Q \times \Sigma^*)$, definiert durch

$$(q, w) \xrightarrow{M} (p, x) \iff w = ax, a \in \Sigma \text{ und } \delta(q, a) = p.$$

Reflexive und transitive Hülle davon:

$$\xrightarrow{*}_M$$

Nichtexistenzbeweise

☐ Direkt (Lemma 3.3)

Lemma:

Sei A ein EA, $x, y \in \Sigma^*$, $x \neq y$

Sei $\delta^*(q_0, x) = \delta^*(q_0, y)$

Dann gilt $xz \in L(A) \iff yz \in L(A)$

Widerspruchsbeweis:

☐ Für einen Widerspruch nehmen wir an L sei regulär

☐ $\Rightarrow A = (Q, \Sigma, \delta_A, q_0, F)$ existiert mit $L(A) = L$

☐ Wir suchen mindestens $Q+1$ wörter w_i die nicht im selben zustand landen

Wir suchen für jedes w_i ein z_i so dass $w_i \in L$ aber für alle anderen $w_j \notin L$

☐ $\Rightarrow$ Widerspruch da für mindestens einen Zustand $\delta^*_A(q_0, w_i) = \delta^*_A(q_0, w_j)$ gelten muss (Lemma)

☐ $\Rightarrow$ L ist nicht regulär

☐ Pumping Lemma für reguläre Sprachen (Lemma 3.4)

Lemma:

Sei L regulär

Es existiert $n_0 \in \mathbb{N}$ so dass für jedes $w \in \Sigma^*$ mit $|w| \geq n_0$ gilt:

$w = yxz$ (lässt sich zerteilen) wobei

i) $|yx| \leq n_0$

ii) $|x| \geq 1$

iii) entweder $\{y(x^k)z \mid k \in \mathbb{N}\} \subseteq L$ oder $\{y(x^k)z \mid k \in \mathbb{N}\} \cap L = \emptyset$

Widerspruchsbeweis:

☐ Für einen Widerspruch nehmen wir an L sei regulär

☐ $\Rightarrow$ Wir suchen ein Wort w mit $|w| \geq n_0$ so dass egal wie wir y, x und z wählen $y(x^k)z$ für mindestens ein k in und ein k nicht in L ist

☐ $\Rightarrow$ Widerspruch

☐ $\Rightarrow$ L ist nicht regulär

☐ Kolmogorov (Satz 3.1)

Satz:

Sei $L \subseteq \{0,1\}^*$ eine reguläre Sprache. Sei $L_x = \{y \in \{0,1\}^* \mid xy \in L\}$ für jedes $x \in \{0,1\}^*$. Dann existiert eine Konstante c , so dass für alle $x, y \in \{0,1\}^*$

$K(y) \leq \lceil \log_2(n+1) \rceil + c$, falls y das n -te Wort in L_x ist.

Widerspruchsbeweis:

☐ Für einen Widerspruch nehmen wir an L sei regulär

☐ Wir suchen eine Sprache L_x wo wir das erste Wort w_1 in L_x bestimmen können, x soll von n abhängen

☐ $K(w_1) \leq \lceil \log_2(1+1) \rceil + c = k$ für $k = 1 + c$

☐ Widerspruch da die Anzahl Programme kürzer oder gleich k endlich ($\max 2^k$) sind die Menge $\{x(n) \mid n \in \mathbb{N}\}$ aber unendlich ist

☐ $\Rightarrow$ L ist nicht regulär

Mindestanzahl Zustände

Wir zeigen dass ein EA der L akzeptiert mindestens x Zustände haben muss:

☐ Wir suchen x Präfixe die in unterschiedlichen Zuständen landen (aus

graphischer Darstellung leicht ablesbar (Weg zum Zustand))

- ☐ Wir zeigen in einer Tabelle für jede Kombination von diesen Präfixen, dass es einen Suffix gibt so dass einer von beiden akzeptiert wird, der andere nicht
- ☐ Deshalb sind alle Präfixe in unterschiedlichen Klassen
- ☐ => Es gibt mindestens x Zustände

Nichtdeterministische Endliche Automaten (NEA)

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : endliche Zustandsmenge

Σ : Eingabealphabet

δ : Übergangsfunktion $\delta : Q \times \Sigma \rightarrow P(Q)$, also Menge möglicher Folgezustände

q_0 : Startzustand

F : Menge akzeptierender Zustände

Potenzmengenkonstruktion

Satz 3.2. Zu jedem NEA M existiert ein EA A , so dass

$$L(M) = L(A).$$

Beweis. Sei $M = (Q, \Sigma, \delta_M, q_0, F)$ ein NEA. Wir konstruieren einen EA $A = (Q_A, \Sigma_A, \delta_A, q_{0A}, F_A)$ wie folgt:

- (i) $Q_A = \{\langle P \rangle \mid P \subseteq Q\}$,
- (ii) $\Sigma_A = \Sigma$,
- (iii) $q_{0A} = \{\langle q_0 \rangle\}$,
- (iv) $F_A = \{\langle P \rangle \mid P \subseteq Q \text{ und } P \cap F \neq \emptyset\}$,
- (v) δ_A ist eine Funktion von $Q_A \times \Sigma_A$ nach Q_A , die wie folgt definiert ist. Für jedes $\langle P \rangle \in Q_A$ und jedes $a \in \Sigma_A$ ist

$$\begin{aligned}\delta_A(\langle P \rangle, a) &= \left\langle \bigcup_{p \in P} \delta_M(p, a) \right\rangle \\ &= \{\langle q \in Q \mid \exists p \in P, \text{ so dass } q \in \delta_M(p, a) \rangle\}.\end{aligned}$$

Turingmaschinen

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

Q : endliche Menge von Zuständen

Σ : Eingabealphabet (ohne Blank-Zeichen)

Γ : Bandalphabet (enthält Σ und das Blank-Symbol $\sqcup$)

δ : Übergangsfunktion $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, N\}$

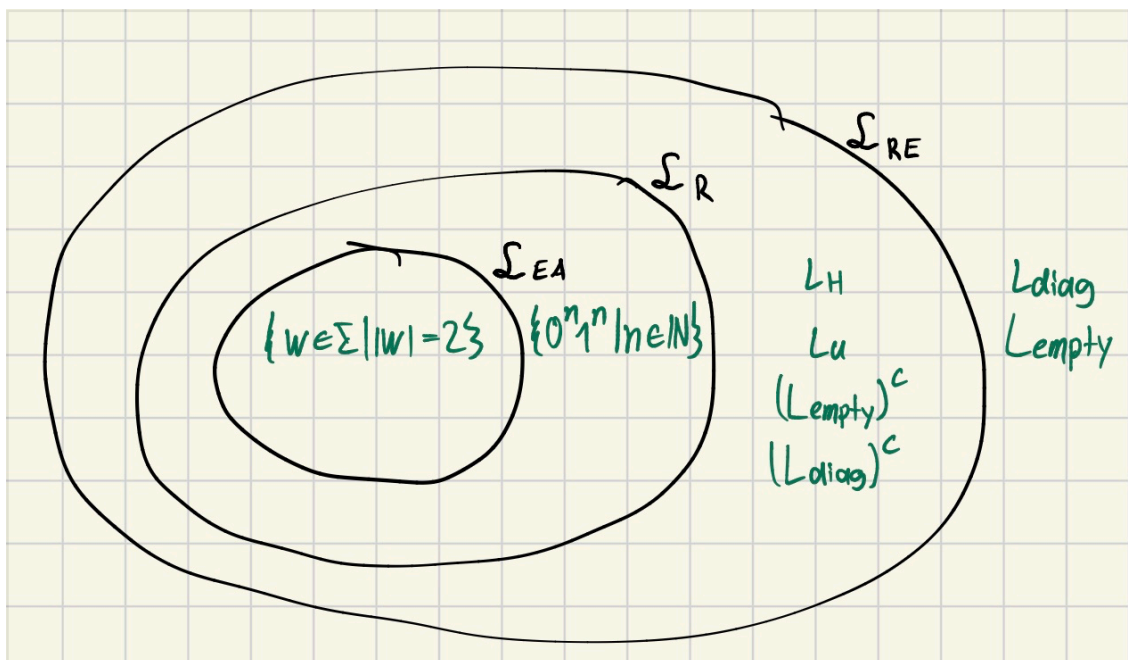
q_0 : Startzustand

$q_{\text{accept}}, q_{\text{reject}}$: akzeptierender bzw. verwerfender Zustand

Γ ein Alphabet, **Arbeitsalphabet** genannt, wobei $\Sigma \subseteq \Gamma$, $\epsilon, \sqcup \in \Gamma$, $\Gamma \cap Q = \emptyset$,

$\delta : (Q - \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, N\}$

Klassen von Sprachen nach Berechenbarkeit



- ☐ $L_H = \{\text{Kod}(M)\#w \mid w \in \{0,1\}^* \text{ und } M \text{ hält auf } w\}$
- ☐ $L_U = \{\text{Kod}(M)\#w \mid w \in (\Sigma_{\text{bool}})^* \text{ und } M \text{ akzeptiert } w\}$ universelle Sprache
- ☐ $L_{\text{empty}} = \{\text{Kod}(M) \mid L(M) = \emptyset\}$
 - ☐ $(L_{\text{empty}})^c = \{w \in (\Sigma_{\text{bool}})^* \mid w \neq \text{Kod}(M) \text{ für alle TM } M \text{ oder } w = \text{Kod}(M) \text{ und } L(M) \neq \emptyset\}$
- ☐ $L_{\text{diag}} = \{w \in (\Sigma_{\text{bool}})^* \mid w = w_i \text{ für ein } i \in \mathbb{N} - \{0\} \text{ und } M_i \text{ akzeptiert } w_i \text{ nicht}\}$
- ☐ $(L_{\text{diag}})^c = \{w \in (\Sigma_{\text{bool}})^* \mid w = w_i \text{ für ein } i \in \mathbb{N} - \{0\} \text{ und } M_i \text{ akzeptiert } w_i\}$

Diagonalisierung

L_diag Beispiel:

1. Nummeriere alle Turingmaschinen als

$M_1, M_2, M_3, \dots$

und alle Wörter über $\Sigma_{bool} = \{0, 1\}$ in kanonischer Reihenfolge als

$w_1, w_2, w_3, \dots$ Buch Theoretische Informatik

2. Definiere eine unendliche 0/1-Matrix $A = (d_{ij})_{i,j \geq 1}$ durch

$$d_{ij} = 1 \iff M_i \text{ akzeptiert } w_j.$$

Die i -te Zeile kodiert also genau die Sprache $L(M_i)$:

$$L(M_i) = \{w_j \mid d_{ij} = 1\}. \quad \text{Buch Theoretische Informatik}$$

3. Definiere die Diagonalsprache

$$L_{\text{diag}} := \{w_i \mid M_i \text{ akzeptiert } w_i \text{ nicht}\} = \{w_i \mid d_{ii} = 0\}.$$

4. Angenommen, **zum Widerspruch**, $L_{\text{diag}} \in L_{RE}$.

Dann gibt es eine Turingmaschine M mit $L(M) = L_{\text{diag}}$.

Da jede TM irgendwo in der Aufzählung vorkommt, existiert ein Index k mit $M = M_k$.

5. Betrachte das Wort w_k .

Aus der Definition von L_{diag} folgt:

$$w_k \in L_{\text{diag}} \iff M_k \text{ akzeptiert } w_k \text{ nicht} \iff w_k \notin L(M_k).$$

Da aber $L(M_k) = L_{\text{diag}}$ gilt, bekämen wir

$$w_k \in L(M_k) \iff w_k \notin L(M_k),$$

ein Widerspruch.

6. Also war die Annahme falsch. Daher gilt:

$$L_{\text{diag}} \notin L_{RE}.$$

Die diagonalisierung kann z.B. auch genutzt werden um zu zeigen dass das Intervall $(0,1)$ überabzählbar ist indem man eine Bijektion annimmt. (Zeilen: N, Spalten i-te Nachkommastelle)